

postgres에서 t_user 테이블에
username, password 칼럼 추가하는 SQL 줘

2초 동안 생각함 >

```
ALTER TABLE t_user  
ADD COLUMN username VARCHAR(100)
```

</> SQL



보통은 password 에 **비밀번호 원문**을 저장하면 안 되고, 해시값을 저장해야 하므로 이름을 이렇게 하는 게 더 좋습니다.

```
ALTER TABLE t_user  
ADD COLUMN username VARCHAR(100)
```

</> SQL



중복 아이디 방지도 하려면:

```
ALTER TABLE t_user  
ADD COLUMN username VARCHAR(100) UNIQUE
```

</> SQL



필수 입력까지 강제하려면:

```
ALTER TABLE t_user  
ADD COLUMN username VARCHAR(100) NOT NULL UNIQUE
```

</> SQL



이미 기존 데이터가 있는 테이블이면 NOT NULL 추가 시 에러 날 수 있어서, 처음에는 NOT NULL 없이 추가한 뒤 데이터를 채우고 나중에 제약조건을 거는 게 안전합니다.



내가 react native expo랑 nodejs hono typescript 로 채팅 만들고 있어

아직은 1:1 채팅만 지원하는데,
나중엔 gpt codex 같은놈이랑 카톡같이 만들고 싶어(훗날)
그런데 DB는 잘못 설계하면 골치 아프잔아?
니가 향후까지 생각해서 테이블 어떻게 하면 좋을지 검토 좀 해줘.

내가 생각한건 여기까지야.
DB는 postgres 쓰고있어.

더 보기 ∨

지금 설계는 "1:1 DM만 당장 저장"은 가능하지만, 나중에 카톡방/AI봇/그룹채팅/읽음처리
로 확장하려면 메시지 테이블에 받는 사람을 직접 넣는 구조가 금방 막힙니다. 핵심은 **채팅방 테이블 + 참여자 테이블 + 메시지 테이블**로 나누는 겁니다.

4s 동안 생각함 >

지금 생각한 구조:

```
t_chat_message(
  id SERIAL PRIMARY KEY,
  message VARCHAR,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  sender_user_id INT4,
  recieve_user_id INT4
```

</> SQL



이 구조는 처음엔 쉬운데, 나중에 문제가 생깁니다.

1. 현재 구조의 문제

문제 1. 1:1 채팅방 개념이 없음

예를 들어 A와 B가 대화하면 메시지는 이렇게 쌓입니다.

```
A -> B : 안녕
B -> A : 반가워
A -> B : 뭐해?
```



그런데 DB에는 "A와 B의 대화방"이라는 개념이 없습니다.

그래서 대화 목록을 만들 때 매번 이런 식으로 조회해야 합니다.

```
WHERE
  (sender_user_id = A AND recieve_user_id = B)
OR
</> SQL
```



처음엔 괜찮지만 나중에 복잡해집니다.

문제 2. 그룹채팅으로 확장 불가

나중에 단독방을 만들면 메시지 하나에 받는 사람이 여러 명입니다.

```
A -> B, C, D
```



그런데 recieve_user_id 는 한 명만 저장할 수 있습니다.

문제 3. GPT/Codex 같은 AI 채팅 상대 넣기 애매함

AI도 결국 "채팅방 참여자"처럼 다루는 게 좋습니다.

예를 들면:

```
채팅방 1
- 나
- GPT봇
```



또는:

```
채팅방 2
- 나
- 친구
- GPT봇
```



이렇게 확장하려면 sender_user_id , recieve_user_id 방식보다 room_id 방식이 훨씬 좋습니다.

추천 구조

처음부터 이렇게 가는 게 좋습니다.

```
t_user
t_chat_room
t_chat_room_member
t_chat_message
```



핵심 구조는 이겁니다.

```
사용자
↓
채팅방 참여자
↓
채팅방
↓
메시지
```



1. 사용자 테이블

이미 `t_user` 가 있다고 했으니 예시는 이렇게만 보겠습니다.

```
CREATE TABLE t_user (
  id SERIAL PRIMARY KEY,
  username VARCHAR(100) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
```

</> SQL



2. 채팅방 테이블

```
CREATE TABLE t_chat_room (
  id SERIAL PRIMARY KEY,
  room_type VARCHAR(20) NOT NULL DEFAULT 'direct',
  title VARCHAR(100),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

</> SQL



`room_type` 은 이런 식으로 씁니다.

```
direct  : 1:1 채팅
group   : 단체 채팅
ai       : AI와의 채팅
```



예시:

```
id | room_type | title
---|-----|-----
1  | direct    | null
2  | group     | 스터디방
3  | ai        | GPT 상담방
```



3. 채팅방 참여자 테이블

이게 중요합니다.

```
CREATE TABLE t_chat_room_member (
  id SERIAL PRIMARY KEY,
  room_id INT NOT NULL REFERENCES t_chat_room(id) ON DELETE CASCADE,
  user_id INT NOT NULL REFERENCES t_user(id) ON DELETE CASCADE,
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_read_message_id INT,
  UNIQUE (room_id, user_id)
```

</> SQL



이 테이블은 "누가 어떤 채팅방에 들어가 있는가"를 저장합니다.

예를 들어 1:1 채팅이면:

```
room_id | user_id
-----|-----
1       | 10
1       | 20
```



그룹채팅이면:

```
room_id | user_id
-----|-----
2       | 10
2       | 20
2       | 30
2       | 40
```



나중에 AI도 t_user 에 하나의 유저처럼 넣으면 됩니다.

```
id | username
---|-----
999 | gpt_bot
```



그러면 AI 채팅방도 그냥 똑같이 처리됩니다.

```
room_id | user_id
-----|-----
3       | 10
3       | 999
```



4. 메시지 테이블

```
CREATE TABLE t_chat_message (
  id SERIAL PRIMARY KEY,
  room_id INT NOT NULL REFERENCES t_chat_room(id) ON DELETE CASCADE,
  sender_user_id INT NOT NULL REFERENCES t_user(id) ON DELETE CASCADE,
  message TEXT NOT NULL,
  message_type VARCHAR(20) NOT NULL DEFAULT 'text',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

</> SQL



여기서 중요한 건 recieve_user_id 를 없앤다는 겁니다.

왜냐하면 메시지는 특정 사람에게 가는 게 아니라 **채팅방에 올라가는 것**으로 보는 게 좋습니다.

```
메시지 → 채팅방
채팅방 → 참여자들
```



이 구조입니다.

최종 추천 SQL

```
CREATE TABLE t_chat_room (
  id SERIAL PRIMARY KEY,
  room_type VARCHAR(20) NOT NULL DEFAULT 'direct',
```

```
title VARCHAR(100),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE t_chat_room_member (
  id SERIAL PRIMARY KEY,
  room_id INT NOT NULL REFERENCES t_chat_room(id) ON DELETE CASCADE,
  user_id INT NOT NULL REFERENCES t_user(id) ON DELETE CASCADE,
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_read_message_id INT,
  UNIQUE (room_id, user_id)
);

CREATE TABLE t_chat_message (
  id SERIAL PRIMARY KEY,
  room_id INT NOT NULL REFERENCES t_chat_room(id) ON DELETE CASCADE,
  sender_user_id INT NOT NULL REFERENCES t_user(id) ON DELETE CASCADE,
  message TEXT NOT NULL,
  message_type VARCHAR(20) NOT NULL DEFAULT 'text',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

조회 성능용 인덱스도 추가

채팅은 메시지 조회가 많기 때문에 인덱스를 추가하는 게 좋습니다.

```
CREATE INDEX idx_chat_message_room_created
ON t_chat_message(room_id, created_at);

CREATE INDEX idx_chat_room_member_user
ON t_chat_room_member(user_id);

CREATE INDEX idx_chat_room_member_room
```

</> SQL



메시지 불러오기 예시

특정 채팅방 메시지 조회

```
SELECT
  m.id,
```

```
m.message,  
m.message_type,  
m.created_at,  
u.id AS sender_user_id,  
u.username AS sender_username  
FROM t_chat_message m  
JOIN t_user u ON m.sender_user_id = u.id  
WHERE m.room_id = 1
```

</> SQL



내가 들어간 채팅방 목록 조회

```
SELECT  
  r.id AS room_id,  
  r.room_type,  
  r.title,  
  r.created_at  
FROM t_chat_room r  
JOIN t_chat_room_member rm ON r.id = rm.room_id  
WHERE rm.user_id = 10
```

</> SQL



나중에 카톡처럼 만들려면 추가할 수 있는 것들

처음부터 다 만들 필요는 없지만, 확장 가능하게 생각하면 좋습니다.

메시지 삭제/수정

```
ALTER TABLE t_chat_message  
ADD COLUMN deleted_at TIMESTAMPTZ
```

</> SQL



실제로 DB에서 바로 삭제하지 않고 deleted_at 만 찍는 방식입니다.

이미지/파일 메시지

나중에 이미지 전송을 하려면 이렇게 추가할 수 있습니다.


```
CREATE TABLE t_chat_attachment (
  id SERIAL PRIMARY KEY,
  message_id INT NOT NULL REFERENCES t_chat_message(id) ON DELETE CASCADE,
  file_url TEXT NOT NULL,
  file_type VARCHAR(50),
  file_name VARCHAR(255),
  file_size INT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

</> SQL



그러면 메시지는 이렇게 됩니다.

```
message_type = 'image'
message = '사진을 보냈습니다.'
```



실제 파일 정보는 t_chat_attachment 에 저장합니다.

가장 현실적인 첫 버전

지금 당장 Expo + Hono + PostgreSQL로 1:1 채팅만 만들 거면, 최소한 이 정도로 시작하면 됩니다.

```
CREATE TABLE t_chat_room (
  id SERIAL PRIMARY KEY,
  room_type VARCHAR(20) NOT NULL DEFAULT 'direct',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE t_chat_room_member (
  id SERIAL PRIMARY KEY,
  room_id INT NOT NULL REFERENCES t_chat_room(id) ON DELETE CASCADE,
  user_id INT NOT NULL REFERENCES t_user(id) ON DELETE CASCADE,
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  UNIQUE (room_id, user_id)
);

CREATE TABLE t_chat_message (
  id SERIAL PRIMARY KEY,
  room_id INT NOT NULL REFERENCES t_chat_room(id) ON DELETE CASCADE,
  sender_user_id INT NOT NULL REFERENCES t_user(id) ON DELETE CASCADE,
  message TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

이게 가장 무난합니다.

결론

`sender_user_id`, `recieve_user_id` 를 메시지 테이블에 직접 넣는 방식은 초보 예제에서는 쉽지만, 실서비스 구조로 가면 불편합니다.

추천은 이 구조입니다.

<code>t_chat_room</code>	채팅방
<code>t_chat_room_member</code>	채팅방 참여자
<code>t_chat_message</code>	메시지



그리고 메시지는 이렇게 생각하면 됩니다.

누가 누구에게 보냈다



보다는

누가 어느 채팅방에 메시지를 올렸다



이렇게 설계하는 게 좋습니다.

